



Referendum Binario Universitario

Project Work 2026

Algoritmi e Protocolli per la sicurezza

6 Settembre 2026

Gruppo 1

Componenti del gruppo

Aniello De Luca - IE22700071

Daniele De Luca - IE22700125

Sommario

WP 1: Modello	4
1.1 - Introduzione	4
1.2 - Funzionalità e tipo di voto	4
1.3 - Attori e obiettivi	4
1.3.1 - Elettore (Studente)	4
1.3.2 - Sistema di Autenticazione (Identity Provider, IdP)	4
1.3.3 - Autorità Elettorale per la Raccolta (Server Centrale)	4
1.3.4 - Commissione Elettorale (Tallying Authority)	4
1.3.5 - Autorità di Mix	5
1.4 - La scheda elettorale come asset da proteggere	5
1.5 - Threat model	5
1.5.1 - Intercettazione e modifica	5
1.5.2 - Avversario de-anonimizzatore	5
1.5.3 - Elettore disonesto	6
1.5.4 - Coercizione e compravendita del voto	6
1.5.5 - Compromissione parziale del server (insider)	6
1.5.6 - Autorità finali disoneste	6
1.6 - Proprietà di sicurezza e resilienza	6
1.6.1 - Autenticità	6
1.6.2 - Unicità	6
1.6.3 - Segretezza (pseudoanonimato)	6
1.6.4 - Integrità	6
1.6.5 - Verificabilità individuale	7
1.6.6 - Verificabilità universale	7
1.6.7 - Ricevuta e coercizione	7
1.6.8 - Trasparenza	7
1.6.9 - Efficienza	7
1.7 - Revocabilità e sostituzione del voto	7
1.8 - Compromessi e prioritizzazione dei requisiti	8
1.9 - Assunzioni di fiducia e rischio residuo	8
WP 2: Soluzione	10
2.1 - Panoramica del protocollo	10
2.1.1 - Ruoli e abbreviazioni	10
2.2 - Primitive crittografiche e infrastruttura di fiducia	10
2.2.1 - Infrastruttura a chiave pubblica	11
2.2.2 - Firme digitali	11
2.2.3 - Cifratura ibrida	11
2.2.4 - Funzioni di hash e Merkle tree	11
2.2.5 - Custodia distribuita con Shamir	11
2.3 - Configurazione, gestione delle chiavi e manifesto	12
2.3.1 - Generazione e uso delle chiavi	12
2.3.2 - Custodia della chiave di scrutinio	12
2.3.3 - Autenticazione delle chiavi pubbliche	13
2.3.4 - Manifesto della consultazione	13
2.4 - Oggetti e messaggi del protocollo	13
2.5 - Protocollo operativo	16
2.5.1 - Fase 0: Avvio della Consultazione	16
2.5.2 - Fase 1: Registrazione e autenticazione	17
2.5.3 - Fase 2: Preparazione e invio della scheda	17
2.5.4 - Fase 3: Accettazione e registrazione	17
2.5.5 - Fase 4: Chiusura e finalizzazione della bacheca	18

2.5.6 - Fase 5: Mix.....	18
2.5.7 - Fase 6: Scrutinio e pubblicazione.....	18
2.6 - Procedure di verifica.....	19
2.6.1 - Verifica individuale.....	19
2.6.2 - Verifica pubblica.....	19
2.7 - Regole operative, revocabilità ed efficienza.....	20
2.7.1 - Accettazione e revocabilità.....	20
2.7.2 - Efficienza.....	20
WP 3: Analisi della sicurezza.....	21
3.1 - Impostazione dell'analisi e assunzioni.....	21
3.2 - Analisi delle proprietà di sicurezza.....	21
3.2.1 - Autenticità, unicità e integrità.....	21
3.2.2 - Segretezza e collusioni.....	22
3.2.3 - Verificabilità del voto e del risultato.....	22
3.2.4 - Ricevuta, coercizione e fiducia.....	23
3.3 - Analisi rispetto al threat model.....	23
3.3.1 - Attacchi di rete e correlazione dei dati.....	23
3.3.2 - Elettore, coercizione e dispositivo.....	23
3.3.3 Collector, Mix e Commissione.....	24
3.4 - Valutazione delle alternative e dei compromessi.....	24
WP 4: Implementazione e prestazioni.....	26
4.1 - Scelte implementative e architettura.....	26
4.2 - Struttura dei moduli e del codice.....	26
4.2.1 - Primitive e messaggi.....	26
4.2.2 - Attori e consultazione.....	26
4.2.3 - Raccolta e verifiche.....	27
4.2.4 - Interfaccia e sperimentazione.....	27
4.3 - Sperimentazione e verifica del funzionamento.....	27
4.3.1 - Esecuzione del flusso completo.....	27
4.3.2 - Scenari di rifiuto.....	28
4.3.3 - Test automatici.....	28
4.4 - Valutazione delle prestazioni.....	28
4.4.1 - Metodo di misurazione.....	28
4.4.2 - Risultati del benchmark.....	29
4.4.3 - Discussione dei risultati.....	31

WP 1: Modello

1.1 - Introduzione

Il presente documento definisce i requisiti di sicurezza per un sistema di voto elettronico destinato a un referendum binario universitario "Sì/No". La soluzione digitale mira a semplificare lo scrutinio e favorire la partecipazione, ma richiede di chiarire quali dati proteggere e da quali minacce.

In questo capitolo definiamo le esigenze del sistema e i compromessi tra i requisiti, in particolare tra la segretezza della preferenza e la verificabilità dei risultati. Per segretezza intendiamo la protezione del legame tra identità e preferenza rispetto a ogni singola entità del sistema, sotto le assunzioni dichiarate in 1.9.

1.2 - Funzionalità e tipo di voto

Il progetto propone un sistema di voto elettronico per un referendum binario universitario, in cui gli studenti possono esprimere la propria preferenza scegliendo esclusivamente tra «Sì» e «No». La scelta binaria mantiene semplici la rappresentazione della preferenza, i controlli di validità e lo scrutinio, contribuendo a contenere il costo delle operazioni. Resta comunque necessario riconoscere ed escludere le schede malformate.

Lo studente si autentica e il sistema verifica che abbia diritto a partecipare alla consultazione. L'elettore può quindi accedere all'interfaccia di voto, selezionare e inviare la propria preferenza. L'operazione si conclude con la ricezione di una conferma che attesta l'avvenuta registrazione senza indicare la scelta effettuata.

1.3 - Attori e obiettivi

Il modello comprende cinque attori.

1.3.1 - Elettore (Studente)

È lo studente con diritto al voto, cioè iscritto all'università e ammesso alla consultazione. Vuole esprimere la propria preferenza in modo autonomo, preservando lo pseudoanonimato rispetto ai singoli gestori del sistema. Ha bisogno di poter verificare che il proprio voto sia stato registrato e incluso nel processo di conteggio, senza che tale verifica riveli la preferenza. La conferma ufficiale rilasciata dal sistema, considerata da sola, non deve dimostrare a terzi il contenuto del voto.

1.3.2 - Sistema di Autenticazione (Identity Provider, IdP)

È l'entità che verifica l'identità degli aventi diritto. Il suo compito è autenticare gli studenti in modo certo e abilitare al voto chi ne ha diritto, una sola volta. Deve restare logicamente separato dalla raccolta delle schede, perché se la stessa entità disponesse contemporaneamente delle informazioni sull'identità e sul contenuto del voto, la segretezza dipenderebbe dalla sola correttezza del gestore.

1.3.3 - Autorità Elettorale per la Raccolta (Server Centrale)

È l'infrastruttura che riceve e conserva le schede. Le registra in un registro pubblico, la "bacheca elettorale", strutturato in modo che alterazioni, sostituzioni o rimozioni successive alla registrazione siano rilevabili attraverso i controlli pubblici. Rilascia inoltre una prova di avvenuta ricezione per ciascuna scheda accettata.

1.3.4 - Commissione Elettorale (Tallying Authority)

È l'ente che esegue lo scrutinio a urne chiuse e pubblica il risultato. La fase finale deve richiedere la partecipazione di più membri della Commissione, in modo che lo scrutinio non dipenda dalla

correttezza di un singolo componente. Il procedimento deve inoltre consentire controlli pubblici sulla coerenza del risultato prodotto.

1.3.5 - Autorità di Mix

È un'autorità distinta dal server di raccolta e dalla Commissione elettorale. Interviene fra la raccolta e lo scrutinio con l'obiettivo di separare i riferimenti delle schede registrate dalle informazioni rese disponibili a chi conta i voti. Non deve conoscere né l'identità reale dell'elettore né il contenuto della preferenza.

1.4 - La scheda elettorale come asset da proteggere

L'asset fondamentale del sistema è la scheda elettorale, cioè il dato che rappresenta la preferenza espressa nel contesto della consultazione. Occorre proteggere la preferenza espressa e impedire che venga associata all'identità dell'elettore. La conoscenza della partecipazione o di un riferimento alla scheda non deve permettere, da sola, di ricostruire la scelta.

Gli obiettivi di protezione sono quattro:

1. il contenuto deve restare riservato fino alla chiusura delle urne;
2. la scheda non deve poter essere alterata dopo l'espressione del voto;
3. non deve poter pesare più di una volta;
4. una volta registrata non deve poter essere rimossa senza che l'operazione sia rilevabile.

1.5 - Threat model

L'analisi considera avversari operanti in tempo polinomiale e dotati delle specifiche capacità di seguito descritte. Si prendono in esame attaccanti esterni in grado di intercettare e manipolare il traffico di rete, ma si valutano anche minacce interne, come gestori che tentano di alterare i conteggi o ricostruire l'identità dei votanti.

Si tiene inoltre conto dei rischi legati al fattore umano, includendo sia l'elettore che tenta di frodare il sistema votando più volte, sia soggetti terzi che mirano a influenzare il risultato tramite coercizione o compravendita del voto.

Assumiamo che le primitive crittografiche siano sicure rispetto alle capacità degli avversari descritti.

1.5.1 - Intercettazione e modifica

Un attaccante attivo che controlla la rete può intercettare, leggere, modificare, eliminare o reinviare i messaggi scambiati fra l'elettore e le autorità del sistema. I suoi obiettivi possono essere alterare la scheda prima che venga ricevuta, inserire messaggi fraudolenti oppure eliminare una richiesta in fase di trasferimento per impedirne la registrazione.

1.5.2 - Avversario de-anonimizzatore

Un'entità, eventualmente in collusione con altri gestori del sistema, può cercare di violare lo pseudoanonimato incrociando le informazioni a propria disposizione.

L'avversario può combinare i dati custoditi dai diversi ruoli con quelli pubblicamente disponibili, oppure alterare il proprio comportamento per ottenere informazioni attraverso il risultato. Le possibilità di ricostruire il legame tra identità e preferenza dipendono dai dati accessibili e dalle capacità dei soggetti coinvolti; saranno analizzate nel WP3. Log di accesso, indirizzi IP, orari di votazione e altri metadati possono inoltre consentire correlazioni probabilistiche, distinte dalla ricostruzione crittografica certa del collegamento.

1.5.3 - Elettore disonesto

Un utente legittimamente autenticato può tentare di abusare del sistema.

Può provare a inviare più schede per la stessa elezione, con l'obiettivo di pesare più di una volta sul risultato, oppure negare successivamente di aver partecipato, compromettendo la credibilità dell'infrastruttura.

1.5.4 - Coercizione e compravendita del voto

È lo scenario in cui l'elettore agisce sotto pressione o dietro compenso. L'attaccante e l'elettore possono collaborare e tentare di produrre una prova convincente e trasferibile della preferenza espressa da mostrare al compratore o al coercitore.

1.5.5 - Compromissione parziale del server (insider)

Un gestore disonesto, o un attaccante esterno che ottiene accesso al registro dell'autorità di raccolta, può tentare di leggere, aggiungere, modificare o rimuovere le schede archiviate prima dello scrutinio, cercando di non farsi rilevare dai controlli pubblici. L'obiettivo è manipolare il risultato agendo sulle schede registrate.

1.5.6 - Autorità finali disoneste

Un'autorità di mix disonesta può eliminare, sostituire o manipolare le schede durante la fase di mescolamento. Una Commissione disonesta può invece tentare di pubblicare un risultato che non corrisponde ai voti ricevuti. La collusione fra queste autorità e gli altri gestori può inoltre ridurre o annullare lo pseudoanonimato previsto dal modello.

1.6 - Proprietà di sicurezza e resilienza

Le proprietà seguenti definiscono gli obiettivi di sicurezza del sistema. Le priorità e i compromessi sono discussi in 1.8; il WP3 valuterà quali garanzie la soluzione offre sotto le assunzioni di 1.9 e quali limiti restano.

1.6.1 - Autenticità

Solo gli elettori legittimi, autenticati mediante credenziali di ateneo verificabili, possono partecipare al voto.

1.6.2 - Unicità

Ogni elettore può esprimere al più un voto valido. I tentativi di voto multiplo devono essere scartati prima che possano incidere sul conteggio. Autenticità e unicità insieme rispondono alla minaccia dell'elettore disonesto.

1.6.3 - Segretezza (pseudoanonimato)

Nessuna singola entità del sistema deve poter associare la preferenza espressa all'identità reale dell'elettore.

La protezione è dichiarata rispetto a ciascun gestore considerato singolarmente. La resistenza alla collusione fra più entità non è garantita in modo incondizionato e dipende dall'assunzione di separazione e non collusione dei ruoli. La correlazione di metadati costituisce inoltre un rischio residuo. Questa proprietà risponde alla minaccia dell'avversario de-anonimizzatore.

1.6.4 - Integrità

I voti registrati non devono poter essere alterati, sostituiti o rimossi senza che l'operazione sia rilevabile, e non devono poter comparire schede introdotte senza autorizzazione. La proprietà risponde sia all'attaccante di rete sia alle minacce interne.

1.6.5 - Verificabilità individuale

Ogni elettore deve avere strumenti per controllare autonomamente che la propria scheda abbia seguito correttamente il percorso dal voto fino al conteggio.

In letteratura questa verifica si articola su tre livelli, adottati come riferimento:

- **cast-as-intended:** la scheda preparata deve rappresentare la scelta che l'elettore intende esprimere;
- **recorded-as-cast:** l'elettore può verificare, tramite un riferimento alla propria scheda, che essa sia presente nel registro pubblico senza che il controllo riveli la preferenza;
- **counted-as-recorded:** l'elettore può verificare che quella stessa scheda sia inclusa, senza alterazioni, nel processo di conteggio finale.

Queste verifiche devono essere progettate in modo da non rivelare la preferenza. I tre livelli costituiscono obiettivi del modello; l'analisi di sicurezza specificherà in quale misura il protocollo riesca effettivamente a soddisfarli.

1.6.6 - Verificabilità universale

Il risultato finale deve poter essere verificato pubblicamente e deve corrispondere alle schede registrate. I controlli devono permettere di individuare eventuali incoerenze nei dati pubblicati e nel conteggio. L'analisi di sicurezza distinguerà le garanzie effettivamente ottenute dalle proprietà che restano dipendenti dalle assunzioni di fiducia.

1.6.7 - Ricevuta e coercizione

La ricevuta ufficiale deve permettere all'elettore di controllare la registrazione senza rivelare la preferenza né dimostrarla, da sola, a terzi. Questa garanzia riguarda il contenuto della ricevuta.

La receipt-freeness richiede invece che l'elettore non possa fornire una prova convincente della propria scelta neppure collaborando con un terzo. Un elettore può conservare informazioni utilizzate durante il voto o servirsi di un dispositivo modificato: l'assenza della preferenza nella ricevuta ufficiale non esclude queste possibilità. Nel presente progetto la receipt-freeness completa resta un obiettivo non garantito, così come la resistenza alla coercizione, che comprende anche pressioni e controllo durante il voto.

1.6.8 - Trasparenza

Il sistema non deve dipendere da assunzioni di fiducia implicite o da componenti il cui ruolo nella sicurezza non sia chiaramente identificato. Tutte le assunzioni di fiducia devono essere dichiarate esplicitamente, motivate e analizzate in termini di rischio residuo.

1.6.9 - Efficienza

L'efficienza è un requisito per rendere il sistema utilizzabile sui normali dispositivi degli studenti. L'interfaccia e le procedure di voto devono richiedere risorse contenute e rispondere senza attese eccessive. Lato server, ricezione e registrazione dei dati devono avvenire in tempi rapidi, mantenendo costi compatibili con il numero di partecipanti alla consultazione.

1.7 - Revocabilità e sostituzione del voto

Consideriamo la possibilità per l'elettore di modificare o annullare il proprio voto prima della chiusura delle urne, perché ha implicazioni dirette sulle altre proprietà. Poter votare di nuovo può mitigare la coercizione: l'elettore può sostituire una scelta estorta in un momento non sorvegliato. Permette inoltre di correggere eventuali errori.

Questa possibilità richiede però di stabilire quale scheda sia valida ai fini dell'unicità e di verificare le sostituzioni senza renderle un canale di de-anonimizzazione o una prova della scelta.

Nel presente modello scegliamo di non ammettere la revocabilità: la prima richiesta di voto accettata e registrata è definitiva. L'accettazione non implica che il contenuto della scheda sia già stato riconosciuto come valido; un errore rilevato successivamente non dà diritto a sostituirla. Il compromesso è discusso in 1.8 e il rischio residuo legato alla coercizione in 1.9.

La revocabilità resta quindi una proprietà desiderabile ma esclusa in questa versione e potrebbe essere considerata in un'estensione soltanto se resa compatibile con unicità, segretezza e verificabilità.

1.8 - Compromessi e prioritizzazione dei requisiti

Nel presente modello assegniamo priorità alla segretezza della preferenza e all'unicità del voto, considerati elementi imprescindibili per la validità stessa della consultazione.

La verifica della registrazione deve restare compatibile con la riservatezza della ricevuta. L'elettore deve poter controllare la presenza della propria scheda, mentre la conferma ufficiale non deve dimostrare da sola quale preferenza contiene. Restano i limiti rispetto alla collaborazione dell'elettore e alla coercizione indicati in 1.6.7.

La scelta di rendere definitiva la prima richiesta accettata privilegia un processo semplice da gestire. Il rivoto richiederebbe di gestire le sostituzioni nel registro e selezionare le schede da scrutinare. Rinunciarvi semplifica queste operazioni, ma impedisce di correggere un invio già accettato o sostituire una scelta estorta.

Infine, il compromesso tra trasparenza ed efficienza richiede che l'introduzione di controlli pubblici sul processo mantenga costi compatibili con la scala della consultazione e tempi di risposta adeguati per gli utenti.

1.9 - Assunzioni di fiducia e rischio residuo

In coerenza con la proprietà di trasparenza, dichiariamo qui le principali assunzioni su cui poggia il modello e le conseguenze della loro violazione.

Separazione e non collusione dei ruoli.

Assumiamo che IdP, autorità di raccolta, autorità di mix e Commissione rimangano organizzativamente separate e non colludano in misura sufficiente a ricostruire l'intera relazione identità-voto. La separazione delle responsabilità e la minimizzazione dei dati correlabili, come log, indirizzi IP e orari, riducono le occasioni di collegamento. La sola distinzione organizzativa non impedisce però la condivisione delle informazioni: gli effetti delle collusioni e dei comportamenti attivi restano oggetto dell'analisi nel WP3.

Correttezza dell'IdP.

Assumiamo che l'IdP autentichi correttamente gli aventi diritto e abiliti ciascun elettore al voto una sola volta. Un IdP compromesso potrebbe emettere ulteriori autorizzazioni formalmente valide, violando autenticità e unicità.

Registro pubblico verificabile.

Assumiamo che lo stato finale della bacheca sia pubblicamente disponibile e confrontabile con le prove ottenute dagli elettori. Un'autorità di raccolta disonesta può tentare di modificare o presentare viste differenti del registro; i controlli pubblici permettono di rilevare le incoerenze quando le informazioni pubblicate vengono confrontate.

Scrutinio non affidato a una singola entità.

Assumiamo che lo scrutinio richieda la partecipazione di più membri della Commissione. Una Commissione sufficientemente collusa può comunque pubblicare un risultato scorretto; i controlli pubblici devono rendere rilevabili le incoerenze che il protocollo consente effettivamente di verificare.

Canale, informazioni iniziali di fiducia e dispositivo dell'elettore.

Assumiamo canali che garantiscano autenticità, integrità e riservatezza delle comunicazioni. Le informazioni iniziali necessarie ad autenticare i componenti del sistema provengono da un riferimento affidabile. Assumiamo inoltre che l'elettore disponga almeno al momento della scelta di un dispositivo non compromesso e di un ambiente non sorvegliato.

Un dispositivo malevolo può preparare una scheda diversa dalla scelta mostrata all'elettore, facendo venir meno il cast-as-intended. Un ambiente non sorvegliato consente di votare liberamente, ma non impedisce all'elettore di conservare volontariamente informazioni da mostrare a terzi.

Rischi residui.

Il modello non fornisce anonimato di rete né disponibilità contro attacchi di tipo Denial of Service. Log, indirizzi IP e orari possono essere correlati. La collusione di più autorità può ridurre o annullare lo pseudoanonimato. Avendo escluso la revocabilità, viene inoltre meno la mitigazione che il rivoto potrebbe offrire contro la coercizione.

WP 2: Soluzione

Questo capitolo presenta la soluzione per il modello definito nel WP1: architettura, primitive, distribuzione delle chiavi, oggetti scambiati e azioni delle parti oneste, fino allo scrutinio e alle procedure di verifica.

2.1 - Panoramica del protocollo

Il protocollo assegna a ruoli distinti le principali attività del processo di voto: autenticazione, raccolta, mescolamento e scrutinio. Dopo essersi autenticato presso l'Identity Provider, l'elettore riceve una credenziale pseudonima associata a una chiave temporanea e prepara la propria scheda mediante due livelli di cifratura.

L'Autorità Elettorale per la Raccolta registra la richiesta di voto sulla bacheca pubblica senza poter accedere alla preferenza. L'Autorità di Mix apre soltanto lo strato esterno e modifica l'ordine delle buste interne prima dello scrutinio. La Commissione Elettorale può infine aprire queste ultime e leggere le preferenze secondo le modalità di custodia della chiave descritte nelle sezioni successive.

Nell'esecuzione delle parti oneste, questa separazione distribuisce tra più soggetti le informazioni necessarie a ricostruire il collegamento fra identità e preferenza: nessun ruolo, considerato singolarmente, dispone di entrambi i dati. La protezione di tale collegamento resta condizionata alle assunzioni di separazione e non collusione definite nel WP1.

2.1.1 - Ruoli e abbreviazioni

Nel seguito indichiamo con **E** l'Elettore, con **I** l'Identity Provider, con **C** l'Autorità Elettorale per la Raccolta, denominata anche Collector, con **M** l'Autorità di Mix e con **CE** la Commissione Elettorale.

La soluzione affida la custodia della chiave di scrutinio a tre membri della Commissione, indicati come trustee T_1 , T_2 e T_3 . Le modalità di custodia sono definite nella sezione 2.3.

2.2 - Primitive crittografiche e infrastruttura di fiducia

Le primitive e i meccanismi crittografici adottati sono riepilogati nella tabella 2.1. Nel protocollo si adottano chiavi RSA da 2048 bit.

Tabella 2.1. Primitive crittografiche e impiego nel protocollo

Funzione	Primitiva	Impiego nel protocollo
Firma digitale	RSA-PSS + SHA-256	Credenziali, richieste e attestazioni
Protezione chiavi AES	RSA-OAEP + SHA-256	Busta interna e busta esterna
Cifratura autenticata	AES-256-GCM	Contenuti delle buste e chiave privata di scrutinio
Hash	SHA-256	Riferimenti ai messaggi e Merkle tree finale
Custodia distribuita	Shamir 2 su 3	Quote della chiave K_T

2.2.1 - Infrastruttura a chiave pubblica

La verifica di una firma richiede che la chiave pubblica del firmatario sia stata preventivamente associata al soggetto corretto. Il protocollo assume pertanto una PKI istituzionale mediante la quale i partecipanti autenticano la chiave pubblica di firma della Commissione Elettorale.

La Commissione utilizza questa chiave per firmare il manifesto descritto nella sezione 2.3. Il manifesto distribuisce e associa ai rispettivi ruoli le altre chiavi pubbliche impiegate nella consultazione. La chiave con cui viene verificato non può però essere autenticata dal manifesto stesso: deve essere ottenuta attraverso la PKI o un riferimento istituzionale già fidato. La soluzione non dipende da una specifica implementazione reale della PKI di Ateneo.

2.2.2 - Firme digitali

Le firme digitali sono realizzate con RSA-PSS, SHA-256, MGF1 basata su SHA-256 e un salt casuale di 32 byte. Esse consentono di verificare l'integrità e la provenienza di un oggetto rispetto a una chiave pubblica autenticata.

Per le autorità, una firma valida attribuisce l'oggetto al ruolo associato alla chiave pubblica; per l'elettore, dimostra il possesso della chiave privata temporanea indicata nella credenziale pseudonima.

2.2.3 - Cifratura ibrida

La cifratura ibrida combina AES-256-GCM per il contenuto e RSA-OAEP con SHA-256 per la protezione della chiave AES. Anche OAEP usa MGF1 con SHA-256 e un'etichetta vuota. Per ogni busta si generano una chiave AES di 256 bit e un nonce di 96 bit nuovi e indipendenti; il tag GCM è di 128 bit. La coppia chiave-nonce non viene riutilizzata per cifrare un contenuto diverso. Il contenuto viene cifrato con AES-GCM, mentre la chiave AES viene protetta mediante la chiave pubblica RSA del destinatario. La busta risultante comprende la chiave AES protetta, il nonce, il testo cifrato e il tag di autenticazione.

AES-GCM consente inoltre di autenticare alcuni dati associati, indicati come AAD. Questi dati non vengono cifrati, ma partecipano al calcolo del tag e legano il contenuto alla consultazione, al destinatario e allo scopo previsto per la busta. Durante la decifratura, la verifica ha esito positivo soltanto se il testo cifrato e gli AAD corrispondono a quelli utilizzati originariamente. I valori previsti per la busta interna e per quella esterna sono definiti nella sezione 2.4.

La casualità delle chiavi e delle operazioni di cifratura fa sì che due schede con la stessa preferenza possano produrre buste diverse. Questo è essenziale per il referendum binario: una cifratura pubblica deterministica permetterebbe di cifrare le due alternative e confrontarle con la scheda osservata.

2.2.4 - Funzioni di hash e Merkle tree

SHA-256 viene utilizzato per costruire riferimenti agli oggetti del protocollo e per calcolare i nodi del Merkle tree finale.

Al termine della raccolta, gli hash delle registrazioni ordinate vengono utilizzati per costruire un unico Merkle tree. La radice ne fissa lo stato finale dichiarato dal Collector, mentre le prove di inclusione permettono di verificare la presenza di una voce utilizzando un numero logaritmico di hash, senza acquisire l'intera bacheca.

2.2.5 - Custodia distribuita con Shamir

Lo schema di Shamir con soglia due su tre distribuisce fra tre trustee la custodia della chiave simmetrica *KT*. Questa chiave viene utilizzata per proteggere con AES-256-GCM la chiave privata RSA destinata allo scrutinio.

Una singola quota non è sufficiente per ricostruire *KT*. Nel procedimento previsto, almeno due trustee collaborano per ricostruire *KT* e recuperare la chiave privata completa. Si tratta di custodia distribuita, non di decifratura a soglia eseguita senza ricostruire la chiave.

2.3 - Configurazione, gestione delle chiavi e manifesto

Prima dell'apertura della consultazione, le autorità generano le chiavi necessarie, organizzano la custodia della chiave di scrutinio e autenticano le chiavi pubbliche dei diversi ruoli. La Commissione Elettorale pubblica quindi questi parametri nel manifesto firmato della consultazione.

2.3.1 - Generazione e uso delle chiavi

Le coppie di chiavi sono distinte per ruolo e uso. La loro generazione, come quella delle chiavi AES, dei nonce, dei seriali e della permutazione del Mix, utilizza una sorgente di casualità crittograficamente sicura. Le chiavi private delle autorità restano presso i rispettivi titolari e vengono utilizzate soltanto per lo scopo indicato. La chiave privata di scrutinio è sottoposta alla protezione descritta nella sezione successiva.

Tabella 2.2. Coppie di chiavi e relativo impiego nel protocollo

Soggetto	Coppia di chiavi	Impiego
Identity Provider	skI, pkI	Firma delle credenziali
Collector	(skC, pkC)	Firma delle voci di bacheca, delle ricevute e dello stato finale
Autorità di Mix	(skM, sig, pkM, sig)	Firma del MixRecord
Autorità di Mix	(skM, enc, pkM, enc)	Apertura delle buste esterne
Commissione Elettorale	$(skCE, sig, pkCE, sig)$	Firma del manifesto
Commissione Elettorale	$(skCE, tally, pkCE, tally)$	Apertura delle buste interne durante lo scrutinio
Trustee T_i	(skT, i, pkT, i)	Firma del risultato finale, con $i \in \{1, 2, 3\}$
Elettore	(skE, pkE)	Firma della richiesta di voto

La coppia dell'elettore è temporanea e viene generata localmente nella fase di registrazione. L'elettore conserva skE e comunica all'Identity Provider soltanto pkE , che viene associata alla credenziale pseudonima. Questa coppia è valida esclusivamente per la consultazione in corso.

2.3.2 - Custodia della chiave di scrutinio

La Commissione genera una chiave simmetrica casuale KT di 256 bit e la utilizza con AES-256-GCM per proteggere $skCE, tally$. La copia cifrata della chiave privata viene conservata insieme a un nonce nuovo di 96 bit e al tag di 128 bit. Durante lo sblocco si verifica il tag e si controlla che la chiave recuperata corrisponda alla chiave pubblica di scrutinio indicata nel manifesto.

La Commissione suddivide quindi KT in tre quote mediante lo schema di Shamir con soglia due su tre. Ciascun trustee riceve una sola quota attraverso un canale autenticato e riservato, dopo la verifica del destinatario. Le quote vengono custodite separatamente e non sono pubblicate nel manifesto.

Terminata la distribuzione, KT e $skCE, tally$ non vengono conservate in chiaro. Per rendere nuovamente disponibile la chiave privata di scrutinio sarà quindi necessario ricostruire KT utilizzando le quote di almeno due trustee.

2.3.3 - Autenticazione delle chiavi pubbliche

I partecipanti autenticano $pkCE, sig$ mediante la PKI istituzionale, verificando il legame con la Commissione Elettorale, la validità del certificato e la catena fino al riferimento istituzionale fidato.

L'Identity Provider, il Collector e l'Autorità di Mix trasmettono le proprie chiavi pubbliche alla Commissione attraverso i canali autenticati assunti nel WP1. La Commissione raccoglie anche le chiavi pubbliche di firma dei trustee e verifica la provenienza e lo scopo di ciascuna chiave prima di inserirla nel manifesto.

2.3.4 - Manifesto della consultazione

Il corpo Manifest raccoglie i parametri della consultazione:

$Manifest = (election_id, question, opens_at, closes_at, pkI, pkC, pkM, sig, pkM, enc, pkCE, tally, pkT, 1, pkT, 2, pkT, 3, threshold, rules)$

La Commissione firma l'intero corpo e produce il manifesto firmato:

$\sigma_{CE} = Sign(sk_{CE}, sig, \langle Manifest \rangle, Manifest)$

$SignedManifest = (Manifest, \sigma_{CE})$

election_id identifica univocamente la consultazione e lega a essa credenziali e messaggi. **question** contiene il quesito. Gli istanti sono espressi come secondi interi UTC e devono valere per **opens_at** < **closes_at**. Le nuove richieste sono accettabili quando $opens_at \leq accepted_at < closes_at$.

Le chiavi pubbliche associano ogni operazione al ruolo previsto. **threshold** fissa la soglia di due quote su tre. Le regole sono **FIRST_ACCEPTED** e $revote=false$, ciò significa che la prima richiesta che supera i controlli del Collector è definitiva e non può essere sostituita o revocata. Il recupero della stessa richiesta già registrata non costituisce una nuova accettazione.

La firma della Commissione protegge l'integrità e la provenienza del manifesto. Poiché $pkCE, sig$ è già autenticata mediante la PKI, **SignedManifest** costituisce il riferimento comune per la configurazione della consultazione.

2.4 - Oggetti e messaggi del protocollo

Questa sezione definisce la struttura e il contenuto degli oggetti utilizzati nelle diverse fasi del protocollo. Il suffisso *Body* identifica i campi che costituiscono il corpo di un oggetto prima dell'aggiunta della firma.

Indichiamo con *enc* una codifica univoca e deterministica in JSON, comune a tutti i partecipanti e utilizzata per firme, hash, contenuti cifrati e AAD. Gli oggetti sono rappresentati mediante campi nominati, preservando i tipi dei dati e l'ordine delle sequenze, mentre le formule ne descrivono i contenuti logici. In particolare, la notazione $Sign(sk, X, m)$ indica una firma RSA-PSS di $enc(\langle \text{firma} \rangle, X, m)$, dove *X* è il nome dell'oggetto firmato senza il suffisso *Body*: Manifest, Credential, CastRequest, BoardEntry, Receipt, FinalBoardRoot, MixRecord oppure TallyResult. La verifica usa lo stesso nome e gli stessi dati firmati; per BoardEntry questi ultimi sono costituiti dalla coppia (BoardEntryBody, hB), come specificato sotto.

Ogni soggetto conserva gli oggetti firmati nella versione prodotta o ricevuta. Una ritrasmissione o una nuova consultazione degli stessi dati restituisce quella versione, senza rigenerare firme o cifrature. Poiché alcuni riferimenti includono anche le firme, ricostruire un oggetto con gli stessi campi non basta a preservarne l'hash.

Credential. Dopo aver autenticato **E** e verificato il suo diritto a partecipare alla consultazione, **I** rilascia una credenziale pseudonima che associa un seriale casuale, non ancora assegnato, alla chiave pubblica temporanea pkE :

$$CredentialBody = (election_id, serial, pkE) \quad Credential = (CredentialBody, \sigma I)$$

La firma σI è apposta da **I** sul *CredentialBody* mediante skI e attesta che il seriale e la chiave pubblica pkE sono stati rilasciati per la consultazione identificata da $election_id$. La credenziale non contiene l'identità reale di **E**: come previsto nel WP1, l'associazione tra identità e seriale resta custodita da **I**, mentre la corrispondente chiave privata skE rimane in possesso dell'elettore.

Ballot. È la scheda in chiaro preparata da **E** e contiene l'identificativo della consultazione e la preferenza espressa.

$$Ballot = (election_id, choice)$$

Il campo *choice* appartiene all'insieme $\{0, 1\}$, dove 0 indica “No” e 1 indica “Sì”.

InnerEnvelope. Il *Ballot* viene protetto da **E** mediante uno strato di cifratura destinato alla **CE**. Il contesto nel quale la busta è valida è identificato dai seguenti dati associati autenticati:

$$AADinner = (election_id, CE, \text{«strato interno»})$$

L'etichetta “*strato interno*” distingue la scheda destinata allo scrutinio dallo strato esterno destinato a **M**. La verifica del tag utilizza esattamente *AADinner* nel contesto della consultazione.

$$InnerEnvelope = (encrypted_key, nonce, ciphertext, tag)$$

Il *Ballot* viene cifrato con AES-256-GCM utilizzando una chiave AES e un nonce nuovi. Il campo *encrypted_key* contiene la chiave AES protetta con RSA-OAEP mediante la chiave pubblica di scrutinio $pkCE, tally$; *ciphertext* contiene il *Ballot* cifrato, mentre *tag* ne consente la verifica durante l'apertura della busta.

OuterEnvelope. **E** racchiude la coppia composta da $election_id$ e *InnerEnvelope* in un secondo strato di cifratura destinato a **M**. Per questa operazione vengono utilizzati una nuova chiave AES e un nuovo nonce, indipendenti da quelli della busta interna.

$$AADouter = (election_id, M, \text{«strato esterno»}, HCRENDENTIAL(Credential))$$

Il valore $HCRENDENTIAL(Credential)$ è l'hash della credenziale completa, inclusa la firma σI . La sua presenza negli AAD collega la busta esterna alla specifica credenziale che accompagna la richiesta. Se la credenziale venisse sostituita, il valore degli AAD cambierebbe e la verifica del tag fallirebbe durante l'apertura della busta.

$$OuterEnvelope = (encrypted_key, nonce, ciphertext, tag)$$

Il *ciphertext* e il *tag* sono prodotti cifrando la coppia $(election_id, InnerEnvelope)$ con AES-256-GCM e utilizzando *AADouter*. Il campo *encrypted_key* contiene la nuova chiave AES protetta con RSA-OAEP mediante la chiave pubblica di cifratura di **M**, pkM, enc .

CastRequest. La credenziale e la busta esterna vengono inviate da **E** a **C** mediante una richiesta firmata:

$$CastRequestBody = (election_id, C, Credential, OuterEnvelope) \\ CastRequest = (CastRequestBody, \sigma E) \quad hR = HCAST_REQUEST(CastRequest)$$

La firma σE è apposta mediante skE sul *CastRequestBody*. La sua verifica utilizza la chiave pubblica pkE contenuta nella *Credential* e autenticata dalla firma di *I*. Essa dimostra il possesso della corrispondente chiave privata temporanea senza rivelare pubblicamente l'identità reale dell'elettore. Il riferimento hR è calcolato sulla *CastRequest* completa, inclusa σE .

BoardEntry. Per ogni richiesta accettata, *C* crea una voce della bacheca pubblica, assegnandole un indice progressivo *index*, a partire da 0, e l'istante di accettazione *accepted_at*:

$$\begin{aligned} BoardEntryBody &= (election_id, index, accepted_at, hR, CastRequest) \\ hB &= HBOARD_ENTRY(BoardEntryBody) \quad BoardEntry = (BoardEntryBody, hB, \sigma B) \end{aligned}$$

Il riferimento hB è calcolato esclusivamente sul *BoardEntryBody*. La firma σB è apposta da *C* mediante skC sulla coppia $(BoardEntryBody, hB)$. In questo modo la firma protegge sia il contenuto della registrazione sia il relativo hash, senza introdurre riferimenti circolari. La presenza della *CastRequest* completa permette di verificare pubblicamente la credenziale e le firme già presenti.

Receipt. Dopo la registrazione, *C* restituisce a *E* una ricevuta firmata che collega la richiesta accettata alla corrispondente voce della bacheca:

$$ReceiptBody = (election_id, index, hR, hB) \quad Receipt = (ReceiptBody, \sigma R)$$

La firma σR è apposta da *C* mediante skC sul *ReceiptBody*. La ricevuta non contiene la preferenza in chiaro e attesta esclusivamente l'accettazione e la registrazione della richiesta, non la validità del voto contenuto nelle buste.

FinalBoardRoot. Al termine della raccolta, *C* attesta lo stato finale della bacheca, ordinata secondo l'indice delle registrazioni:

$$\begin{aligned} FinalBoardRootBody &= (election_id, n, root, finalized_at) \\ FinalBoardRoot &= (FinalBoardRootBody, \sigma F) \\ hF &= HFINAL_BOARD_ROOT(FinalBoardRoot) \end{aligned}$$

Il campo n indica il numero complessivo delle voci, $root$ è la radice del Merkle tree e $finalized_at$ è l'istante di finalizzazione, che deve essere maggiore o uguale a *closes_at*. La firma σF è apposta da *C* mediante skC sul *FinalBoardRootBody*. Il riferimento hF è calcolato sulla *FinalBoardRoot* completa, inclusa σF , e identifica lo stato finale della bacheca utilizzato nelle fasi successive.

MerkleProof. Per ciascuna voce, *C* rende disponibile una prova che ne consente la verifica rispetto alla radice finale:

$$MerkleProof = (index, n, percorso\ degli\ hash\ fratelli\ con\ il\ relativo\ ordine\ sinistra/destra)$$

Per $n > 0$, ogni foglia lega l'hash della voce al suo *indice* e al numero complessivo delle registrazioni. I genitori vengono calcolati accoppiando i nodi nell'ordine in cui compaiono:

$$\begin{aligned} leaf_i &= H_MERKLE_LEAF((i, n, hB_i)), \text{ con } 0 \leq i < n \\ parent &= H_MERKLE_NODE((left, right)) \end{aligned}$$

Se un livello contiene un numero dispari di nodi maggiore di uno, l'ultimo viene accoppiato con una copia di sé stesso. Si procede fino a ottenere un solo nodo, che è la radice. Per $n = 1$ la radice coincide con l'unica foglia e il percorso è vuoto; per $n = 0$ si usa $H_MERKLE_EMPTY()$ e non esistono prove di inclusione. La duplicazione serve soltanto al calcolo e non aggiunge registrazioni.

Il verificatore confronta n con il valore firmato e controlla che *index* sia compreso tra 0 e $n-1$. Indice e numero dei nodi del livello determinano l'ordine dei fratelli e la lunghezza del percorso; nei passaggi di duplicazione il fratello deve coincidere con il nodo corrente. Il percorso termina

esattamente alla radice, che deve coincidere con quella nella *FinalBoardRoot*. La prova non richiede una firma propria.

MixRecord. **M** rimuove lo strato esterno delle richieste valide, mescola le *InnerEnvelope* ottenute e produce la lista ordinata risultante.

$$\begin{aligned} L &= (InnerEnvelope_1, \dots, InnerEnvelope_k), \text{ con } k = nout \\ MixRecordBody &= (election_id, hF, nin, nout, eext, HLIST(L)) \\ MixRecord &= (MixRecordBody, L, \sigma M) \quad hM = HMIX_RECORD(MixRecord) \end{aligned}$$

I campi *nin* e *nout* indicano il numero degli ingressi e delle uscite del mix, mentre *eext* conta gli errori rilevati nello strato esterno; la lista *L* contiene quindi *nout* elementi. **M** firma il *MixRecordBody* mediante *skM, sig* e, attraverso *HLIST(L)*, vincola alla firma anche il contenuto e l'ordine della lista, che rimane cifrata per **CE**. Il riferimento *hM* identifica il *MixRecord* completo, inclusi *L* e *σM*. Il protocollo non pubblica la corrispondenza tra ingressi e uscite e non prevede una *shuffle proof*.

TallyResult. La pubblicazione del risultato richiede la partecipazione di almeno due trustee. Quando ci sono buste interne da aprire, le loro quote consentono di recuperare la chiave privata di scrutinio. Terminato il conteggio, **CE** costruisce il corpo del risultato:

$$TallyResultBody = (election_id, hF, hM, No, Sì, nvalid, eext, eint)$$

Tutti i conteggi sono interi non negativi. I campi *No* e *Sì* riportano i rispettivi conteggi e *nvalid* il numero complessivo dei voti validi. Il campo *eext* indica gli errori esterni registrati da **M**, mentre *eint* indica quelli rilevati durante l'elaborazione delle buste interne. I riferimenti *hF* e *hM* collegano il risultato alla *FinalBoardRoot* e al *MixRecord* completi e firmati.

Dopo aver verificato il risultato, almeno due trustee firmano il medesimo *TallyResultBody*. L'insieme *Q* identifica i trustee firmatari:

$$Q \subseteq \{1, 2, 3\}, \text{ con } |Q| \geq 2 \quad TallyResult = (TallyResultBody, \{(i, \sigma T, i) : i \in Q\})$$

Per ogni *Ti* appartenente a *Q*, *σT, i* è la firma apposta mediante *skT, i*. Un *TallyResult* con meno di due firme valide non può essere pubblicato come risultato ufficiale.

2.5 - Protocollo operativo

L'esecuzione utilizza le chiavi e gli oggetti definiti nelle sezioni precedenti. Le comunicazioni avvengono attraverso i canali sicuri assunti nel modello, che garantiscono autenticità, integrità e riservatezza. Nei flussi seguenti, il termine "pubblico" indica la pubblicazione di dati consultabili dagli elettori e dagli altri verificatori.

2.5.1 - Fase 0: Avvio della Consultazione

Completata la configurazione descritta nella sezione 2.3, **CE** pubblica il *SignedManifest*. Prima di partecipare al protocollo, ciascun soggetto autentica la chiave di firma della **CE**, verifica la firma e controlla gli identificativi delle autorità, la distinzione delle chiavi per ruolo e uso, la soglia due su tre, le regole di voto e la finestra temporale definita in 2.3.4. Se i controlli falliscono, l'esecuzione viene interrotta.

I inizializza lo stato delle credenziali rilasciate, mentre **C** predispone la bacheca vuota e l'insieme dei seriali utilizzati. All'istante stabilito nel manifesto, **C** avvia la raccolta delle richieste.

$$CE \rightarrow \text{pubblico}: SignedManifest$$

2.5.2 - Fase 1: Registrazione e autenticazione

E genera localmente la coppia temporanea (skE , pkE) e conserva la chiave privata. Si autentica presso **I** mediante le credenziali istituzionali e comunica $election_id$ e pkE nella stessa sessione autenticata.

I verifica l'identità, il diritto di voto e la consultazione indicata. Il controllo di precedente emissione, l'assegnazione di un seriale casuale non ancora usato, la costruzione e la firma della *Credential* e la sua registrazione avvengono come un'unica operazione, così che due richieste concorrenti non producano due credenziali. **I** conserva la credenziale completa, inclusa la firma, insieme all'associazione tra identità, consultazione, seriale e pkE . Se la stessa identità autenticata ha già una credenziale per la consultazione, **I** restituisce quella conservata soltanto quando pkE coincide. Una chiave diversa non dà luogo a una seconda emissione. Il recupero restituisce la firma originaria; in questa versione non è prevista la sostituzione della chiave temporanea perduta.

E riceve la *Credential*, ne controlla la firma, $election_id$ e la corrispondenza di pkE , quindi conserva l'oggetto ricevuto. L'identificativo e la password istituzionali restano confinati al flusso di autenticazione e non vengono trasmessi a **C**.

$$\begin{aligned} E &\rightarrow I: \text{autenticazione istituzionale, } election_id, pkE \\ I &\rightarrow E: Credential \end{aligned}$$

2.5.3 - Fase 2: Preparazione e invio della scheda

Dopo aver verificato il *SignedManifest* e confermato la propria scelta, **E** costruisce il *Ballot*. Lo inserisce nella *InnerEnvelope* destinata a **CE** e racchiude quest'ultima nella *OuterEnvelope* destinata a **M**, utilizzando i dati associati autenticati definiti nella sezione 2.4.

E inserisce la *Credential* e la *OuterEnvelope* nel *CastRequestBody* e firma il corpo mediante skE . Conserva la *CastRequest* completa prima di inviarla a **C**, così da poter ritrasmettere lo stesso oggetto se la risposta non arriva.

$$E \rightarrow C: CastRequest$$

2.5.4 - Fase 3: Accettazione e registrazione

C utilizza il *SignedManifest* come riferimento per controllare la struttura della *CastRequest*, il destinatario e la coerenza degli identificativi della consultazione. Verifica quindi la firma di **I** sulla *Credential* e, mediante pkE contenuta nella credenziale, la firma di **E** sulla richiesta. Se uno di questi controlli fallisce, la richiesta viene respinta senza aprire le buste e senza produrre una nuova ricevuta.

Se il seriale è già registrato, **C** confronta la richiesta con quella conservata. Quando la *CastRequest* coincide integralmente, restituisce la *BoardEntry* e la *Receipt* esistenti, anche dopo la chiusura; quando è diversa, la respinge. Il recupero non modifica la bacheca, l'istante di accettazione o lo stato del seriale.

Per un seriale non ancora registrato, **C** controlla la finestra temporale del manifesto. Se la raccolta è chiusa, respinge la richiesta. Altrimenti assegna $index$ e $accepted_at$, costruisce e firma la *BoardEntry* e la *Receipt*, registra gli oggetti e marca il seriale come utilizzato in un'unica operazione. Il controllo dello stato e della finestra temporale fa parte della stessa operazione di accettazione. **C** pubblica quindi la *BoardEntry* e restituisce la *Receipt* a **E**. Ogni richiesta diversa che usi quel seriale verrà respinta.

E verifica la firma della *Receipt* e controlla che $election_id$ e hR corrispondano alla consultazione e alla *CastRequest* conservata. Conserva la ricevuta per la verifica di registrazione descritta in 2.6.1.

$$\begin{aligned} C &\rightarrow \text{pubblico}: BoardEntry & C &\rightarrow E: Receipt \end{aligned}$$

2.5.5 - Fase 4: Chiusura e finalizzazione della bacheca

Alla chiusura della raccolta, **C** interrompe l'accettazione di nuove richieste e ordina le *BoardEntry* secondo il relativo *index*. Costruisce quindi l'unico Merkle tree finale applicando la regola definita nella sezione 2.4.

C produce e firma la *FinalBoardRoot*, contenente il numero delle registrazioni, la radice del tree e l'istante di finalizzazione. Conserva e pubblica la *FinalBoardRoot* completa e rende disponibili le *MerkleProof* delle singole voci. Le successive consultazioni restituiscono lo stesso oggetto firmato.

$C \rightarrow \text{pubblico}: \text{FinalBoardRoot e MerkleProof}$

2.5.6 - Fase 5: Mix

Dopo *closes_at*, **M** acquisisce il *SignedManifest*, la bacheca completa e la *FinalBoardRoot*. Autentica il manifesto e svolge i controlli sulla bacheca descritti all'inizio di 2.6.2: struttura delle richieste, firme, destinatario, *election_id*, seriali distinti, indici, istanti dichiarati, hash e radice finale. Per ciascuna *BoardEntry* verificata, **M** ricostruisce AADouter dal contesto della consultazione e dall'hash della *Credential*. Mediante *skM,enc* recupera la chiave AES e apre la *OuterEnvelope*. Controlla il tag, l'*election_id* del contenuto e la struttura della *InnerEnvelope* ottenuta.

I fallimenti della decifratura esterna, del tag o dei controlli sul contenuto appena aperto vengono conteggiati come errori esterni *eext*; le relative buste sono escluse dalla lista. Il numero degli ingressi *nin* coincide con *n*, mentre *nout* conta le *InnerEnvelope* ammesse, con $nin = nout + eext$. Il *Ballot* resta cifrato e non viene ancora validato.

M applica una permutazione casuale alle *InnerEnvelope* ammesse, costruisce la lista e produce il *MixRecord*. Firma e pubblica il record insieme alla lista permutata, mantenendo riservata la corrispondenza tra ingressi e uscite.

$\text{pubblico} \rightarrow M: \text{SignedManifest, bacheca, FinalBoardRoot}$
 $M \rightarrow CE \text{ e pubblico}: \text{MixRecord}$

2.5.7 - Fase 6: Scrutinio e pubblicazione

CE attende *closes_at* e acquisisce il *SignedManifest*, la bacheca, la *FinalBoardRoot* e il *MixRecord*. Controlla lo stato finale della raccolta come **M** e verifica la firma del Mix, gli *election_id*, *hF*, la lunghezza della lista e *HLIST(L)*. Controlla che *nin*, *nout* ed *eext* siano interi non negativi e che $nin = n$ e $nin = nout + eext$. Se uno di questi controlli fallisce, lo scrutinio non prosegue.

Se $nout > 0$, almeno due trustee mettono a disposizione le proprie quote per ricostruire *KT*. **CE** verifica il tag della copia protetta della chiave privata e controlla che la chiave recuperata corrisponda a *pkCE,tally* nel manifesto. Se lo sblocco fallisce o le quote non bastano, lo scrutinio si interrompe: il problema non viene contato come errore di una scheda. Per ciascuna *InnerEnvelope*, **CE** recupera la chiave AES protetta, ricostruisce AADinner e verifica il tag di autenticazione. Il contenuto decifrato viene utilizzato soltanto dopo la verifica del tag; **CE** controlla quindi l'*election_id*, la struttura del *Ballot* e che *choice* sia un intero appartenente a $\{0, 1\}$. Le buste interne o le schede che non superano questi controlli vengono escluse e conteggiate come errori interni, indicati da *eint*.

CE calcola i conteggi *No* e *Si*, determina il numero dei voti validi *nvalid* e controlla la coerenza dei valori ottenuti con *nout*, *eext* ed *eint*. Costruisce quindi il *TallyResultBody*, includendo *hF* e *hM* e copiando *eext* dal *MixRecord* verificato. Quando $nout = 0$, perché la consultazione è vuota o il Mix ha scartato tutte le buste esterne, **CE** non esegue decifrazioni di schede e pone $No = Si = nvalid = eint = 0$, con $eext = n$. Se invece tutte le buste interne falliscono, $eint = nout$ e $No = Si = nvalid = 0$.

Prima di firmare, ciascuno dei trustee firmatari verifica l'elaborazione della stessa lista *L*: controlla le decifrazioni e la classificazione degli errori interni, ricalcola *No*, *Si* e *nvalid* e verifica *eext* e i

riferimenti hF e hM . Almeno due trustee distinti firmano il medesimo *TallyResultBody*. Se le firme valide non bastano, il risultato non viene pubblicato come ufficiale; raggiunto il quorum, **CE** conserva e pubblica il *TallyResult* completo.

$CE \rightarrow \text{pubblico}: \text{TallyResult}$

2.6 - Procedure di verifica

Le procedure seguenti descrivono i controlli che possono essere eseguiti dall'**elettore** e dagli osservatori sui dati conservati o pubblicati durante il protocollo.

2.6.1 - Verifica individuale

E usa la *CastRequest* conservata e la *Receipt* ricevuta o recuperata mediante il reinvio descritto in 2.5.4. Utilizzando l'*index* contenuto nella ricevuta, recupera dalla bacheca la propria *BoardEntry* e la relativa *MerkleProof*. Acquisisce inoltre il *SignedManifest* e la *FinalBoardRoot*.

Il collegamento verificato è il seguente:

$CastRequest \rightarrow Receipt \rightarrow BoardEntry \rightarrow MerkleProof \rightarrow FinalBoardRoot$

Dopo aver verificato il *SignedManifest*, **E** utilizza la chiave pubblica di **C**, contenuta nel manifesto, per verificare le firme della *Receipt*, della *BoardEntry* e della *FinalBoardRoot*. Ricalcola quindi hR a partire dalla *CastRequest* conservata e controlla che coincida con il valore riportato nella *Receipt* e nella *BoardEntry*.

Successivamente, **E** ricalcola hB sul *BoardEntryBody* e lo confronta con i valori contenuti nella *BoardEntry* e nella *Receipt*. Verifica inoltre che *election_id* e *index* coincidano in tutti gli oggetti coinvolti.

E applica la regola *Merkle* di 2.4 a *index*, n e hB , controllando struttura del percorso e corrispondenza con la radice nella *FinalBoardRoot* firmata. Se tutti i controlli hanno esito positivo, la richiesta indicata dalla ricevuta coincide con quella pubblicata ed è inclusa nello stato finale della bacheca.

Per verificare la coerenza dei passaggi successivi, **E** può inoltre eseguire la procedura di verifica pubblica descritta nella sezione seguente.

2.6.2 - Verifica pubblica

Qualsiasi osservatore può ottenere il *SignedManifest*, la bacheca completa, la *FinalBoardRoot*, il *MixRecord* contenente la lista e il *TallyResult*. Dopo aver autenticato la chiave di firma della **CE**, verifica il *SignedManifest* e utilizza le chiavi pubbliche in esso contenute per controllare gli altri oggetti.

Per ogni voce, l'osservatore controlla la struttura della *BoardEntry* e della *CastRequest*, il destinatario **C**, la *Credential* e le firme di **C**, **I** ed **E**. Verifica la coerenza degli *election_id*, l'assenza di seriali duplicati, gli indici consecutivi da 0 a $n-1$ e la condizione $opens_at \leq accepted_at < closes_at$. Ricalcola hR e hB e li confronta con i valori pubblicati.

L'osservatore verifica la firma della *FinalBoardRoot*, il suo *election_id* e $finalized_at \geq closes_at$. Ricostruisce il Merkle tree secondo 2.4 e controlla che numero delle voci e radice coincidano con n e *root* firmati.

Successivamente, verifica il collegamento tra gli oggetti finali:

$FinalBoardRoot \rightarrow MixRecord \rightarrow TallyResult$

Nel *MixRecord*, controlla:

- la firma di **M**;
- il riferimento a hF ;
- il numero di elementi della lista;
- il valore di $HLIST(L)$.

Ricalcola quindi hM sul *MixRecord* completo. Verifica che gli *election_id* siano coerenti e che hF , hM ed *eext* coincidano con i valori riportati nel *TallyResult*. Tutti i conteggi devono essere interi non negativi.

L'osservatore può inoltre confrontare le *InnerEnvelope* pubblicate e segnalare quelle identiche. La segnalazione è diagnostica: non modifica i conteggi, non comporta scarti automatici e non rende negativa da sola la verifica di coerenza.

Infine, l'osservatore verifica che il *TallyResult* contenga almeno due firme valide, apposte da trustee distinti sullo stesso *TallyResultBody*, e controlla le seguenti relazioni:

$$\begin{array}{ll} nin = n & nin = nout + eext \\ nout = nvalid + eint & nvalid = No + Si \end{array}$$

La procedura ha esito positivo soltanto se tutte le firme, i riferimenti, la radice Merkle e le relazioni tra i conteggi risultano coerenti.

2.7 - Regole operative, revocabilità ed efficienza

2.7.1 - Accettazione e revocabilità

La regola *FIRST ACCEPTED* rende definitiva la prima *CastRequest* accettata per un seriale. Una richiesta diversa con lo stesso seriale viene respinta; il reinvio della richiesta originale recupera soltanto gli oggetti già registrati, secondo 2.5.4.

Una richiesta accettata non implica che il *Ballot* contenuto nelle buste sia già stato decifrato e riconosciuto come valido. Se **M** o **CE** rilevano una busta o una scheda non valida nelle fasi successive, la richiesta viene conteggiata rispettivamente tra gli errori esterni o interni, ma il seriale rimane utilizzato. Il protocollo non consente di revocare o sostituire una richiesta già accettata.

2.7.2 - Efficienza

A parametri crittografici fissati, la preparazione e l'accettazione di una singola richiesta richiedono un numero costante di operazioni crittografiche rispetto al numero complessivo n delle richieste.

La costruzione del Merkle tree, il mix, lo scrutinio e la verifica pubblica completa richiedono ciascuno $O(n)$ operazioni. Per $n \geq 2$, una *MerkleProof* contiene $O(\log n)$ hash e la sua verifica richiede $O(\log n)$ passaggi; i casi $n = 0$ e $n = 1$ seguono le regole specificate in 2.4. Bacheca e lista del Mix occupano $O(n)$ spazio a parametri fissati, così come il confronto delle buste interne mediante hash, nel caso atteso.

WP 3: Analisi della sicurezza

3.1 - Impostazione dell'analisi e assunzioni

Il WP1 fissa proprietà e capacità degli avversari; il WP2 definisce i meccanismi su cui valutarle. L'analisi collega ogni requisito ai controlli effettivamente previsti, distinguendo la violazione rilevabile da quella che può produrre dati formalmente coerenti. Le autorità disoneste vengono considerate anche quando il loro comportamento viola una delle assunzioni: in quel caso si determina quale garanzia viene meno.

Si mantengono le ipotesi del WP1 su avversari in tempo polinomiale, sicurezza delle primitive, correttezza dell'IdP, separazione dei ruoli, disponibilità e confrontabilità della bacheca finale, canali sicuri, riferimento iniziale affidabile e dispositivo corretto nel momento assunto. Il WP2 concretizza la fiducia iniziale nella PKI di Ateneo e prescrive la custodia delle chiavi private presso i titolari. Queste condizioni sono assunzioni del modello. In particolare, una firma valida attesta origine e integrità rispetto a una chiave autenticata, ma non dimostra la correttezza delle operazioni dichiarate dal firmatario.

3.2 - Analisi delle proprietà di sicurezza

3.2.1 - Autenticità, unicità e integrità

Autenticità e unicità. L'identità istituzionale viene verificata dall'IdP mediante password e controllo degli aventi diritto. Il Collector autentica invece la richiesta attraverso la credenziale firmata dall'IdP e la firma temporanea su CastRequest. Quest'ultima attesta l'uso di skE associata a pkE , non l'identità reale di chi trasmette il messaggio. Senza le chiavi private pertinenti, un esterno non può alterare i contenuti firmati mantenendo valide le firme; `election_id` e contesto impediscono di riutilizzarle in una consultazione diversa.

Il controllo del seriale, unito all'accettazione indivisibile prevista dal WP2, fa sì che due richieste concorrenti con la stessa credenziale non siano entrambe registrate da un Collector onesto. La cifratura probabilistica non blocca il replay: una copia conserva le firme valide, ma dopo la prima accettazione il seriale è consumato. Il reinvio identico recupera soltanto la registrazione e la ricevuta esistenti; una richiesta diversa viene respinta. *FIRST_ACCEPTED* e `revote=false` rendono definitivo anche un invio la cui busta risulti successivamente non decifrabile. L'emissione indivisibile impedisce duplicazioni concorrenti presso un IdP onesto; l'unicità per persona dipende comunque dalla sua correttezza. Più credenziali con seriali distinti emesse alla stessa identità supererebbero il controllo pubblico di unicità dei seriali. L'unicità della registrazione non prova inoltre che il Mix rispetti le stesse schede nello scrutinio.

Integrità. La firma di CastRequest protegge il corpo prodotto dall'elettore, incluse credenziale e busta esterna. Durante l'apertura, AES-GCM rileva alterazioni rispetto alla chiave e agli AAD attesi; il legame esterno con l'hash della credenziale fa fallire anche il riutilizzo della stessa busta sotto una credenziale diversa. La separazione dei contesti evita di interpretare una firma o un hash come attestazione di un oggetto differente. Queste protezioni non rendono onesto chi possiede una chiave di firma, né impediscono di creare nuove cifrature con una chiave pubblica.

Per la registrazione, hash della richiesta, BoardEntry firmata e Receipt collegano ciò che è stato inviato a ciò che il Collector ha accettato. La radice finale vincola poi hash delle voci, indici e dimensione della bacheca: una modifica non può restare coerente con la stessa radice senza violare la resistenza alle collisioni di SHA-256. Merkle non impedisce però di firmare una seconda radice o di censurare prima della ricevuta. Analogamente, i riferimenti tra FinalBoardRoot, MixRecord e risultato proteggono la coerenza dei dati pubblicati, non la verità delle trasformazioni interne.

3.2.2 - Segretezza e collusioni

Segretezza e pseudoanonimato. Nell'esecuzione fedele del protocollo, la cifratura probabilistica impedisce il riconoscimento delle due preferenze mediante confronto con cifrature di riferimento. L'IdP conosce identità e seriale e, poiché la bacheca è pubblica, riconosce anche la richiesta cifrata associata. Il Collector vede seriale e busta esterna; il Mix ricava la busta interna senza possedere la chiave di scrutinio; la Commissione legge le preferenze delle buste interne permutate, senza ricevere la corrispondenza con i seriali. Il legame protetto è dunque identità-preferenza, non identità-richiesta cifrata.

Per ricostruire direttamente i collegamenti di un'esecuzione fedele, Mix e Commissione con accesso alla chiave di scrutinio sono sufficienti a ottenere seriale-voto: il tratto seriale-busta esterna è già pubblico e non richiede il Collector. L'IdP fornisce identità-seriale; aggiungendolo si ricostruisce identità-voto. IdP e Commissione, senza il collegamento del Mix, non associano direttamente i voti decifrati ai seriali; IdP e Mix, mediante la sola unione dei dati, non aprono la busta interna. Queste sono condizioni di ricostruzione passiva, non soglie universali di resistenza: un Mix attivo può sfruttare il risultato aggregato come mostrato in 3.3.

Il WP2 impone a Mix e Commissione di attendere la chiusura prima di aprire le buste, ma questo resta un vincolo operativo. Due trustee possono ricostruire K_T senza che Shamir imponga una data; inoltre per leggere anticipatamente le richieste ancora a doppia busta servirebbe anche l'apertura esterna del Mix. Il risultato rivela inevitabilmente informazioni: con un solo voto valido o un esito unanime, la preferenza può essere dedotta se è nota la partecipazione al conteggio.

3.2.3 - Verificabilità del voto e del risultato

Verificabilità individuale. Per cast-as-intended, lo stesso dispositivo raccoglie la scelta, costruisce Ballot e lo cifra. Mostrare Sì sullo schermo non prova indipendentemente che sia stato cifrato 1: un dispositivo malevolo può firmare e cifrare correttamente una scelta diversa. Perché la richiesta inviata corrisponda alla scelta, il dispositivo deve restare corretto durante scelta, preparazione e invio. L'ipotesi limitata al solo momento della scelta non basta; i controlli successivi non forniscono una verifica indipendente di questo passaggio.

Recorded-as-cast è la componente maggiormente supportata. Conservando CastRequest e Receipt, l'elettore confronta gli hash con BoardEntry e verifica l'inclusione rispetto a FinalBoardRoot mediante indice, dimensione e cammino Merkle. Un esito positivo lega la richiesta conservata allo stato finale verificato. Una ricevuta che non trova riscontro nella bacheca completa evidenzia invece una mancata registrazione coerente con l'accettazione attestata. Ciò presuppone che i dati siano disponibili e confrontabili; non costringe il Collector ad accettare una richiesta prima di rilasciare la ricevuta.

Per counted-as-recorded, i riferimenti alla radice e al MixRecord consentono di verificare a quale insieme di dati il risultato dichiara di riferirsi. Non permettono di seguire pubblicamente la singola scheda nella permutazione, né di verificare ogni decifratura. L'inclusione nella bacheca è quindi verificabile più direttamente della correttezza del contributo della preferenza al totale.

Verificabilità universale. L'auditor autentica le chiavi tramite PKI e manifesto, verifica firme di credenziali e richieste, seriali distinti, indici, istanti dichiarati, hash delle voci e radice Merkle. Controlla firma del MixRecord, hash e lunghezza della lista pubblicata, riferimenti del risultato e almeno due firme di trustee distinti. Verifica che i conteggi siano interi non negativi e che eext coincida nel MixRecord e nel TallyResult. Le quadrature impongono ingressi = uscite + errori esterni = n , No + Sì = validi e validi + errori interni = uscite. Un hash corretto identifica la lista pubblicata, ma non prova che derivi dagli ingressi; firme e quadrature non provano le classificazioni degli errori o le singole decifrate. La proprietà ottenuta è verificabilità universale parziale, sotto forma di audit pubblico di coerenza.

3.2.4 - Ricevuta, coercizione e fiducia

Receipt-freeness e coercizione. La Receipt contiene riferimenti alla richiesta accettata, senza la preferenza, e non dimostra da sola come l'elettore abbia votato. Un elettore collaborativo può però conservare le chiavi AES e la casualità usata nelle due cifrature RSA-OAEP: un terzo può riprodurre le cifrature ibride e confrontarle con la richiesta registrata, verificando il legame con il Ballot. Un dispositivo controllato può inoltre documentare la preparazione del voto. Il protocollo non impedisce queste prove esterne e non realizza receipt-freeness forte. Con le regole *FIRST_ACCEPTED* e *revote=false*, una richiesta estorta già accettata non può essere sostituita in un momento non sorvegliato.

Trasparenza e fiducia. L'audit espone le attestazioni delle autorità, mentre restano fidati l'IdP per abilitazione ed emissione unica, il Collector per l'accettazione prima della ricevuta, il Mix per la corrispondenza delle buste e la Commissione per decifrare e conteggio. I trustee custodiscono le quote e approvano il risultato; la PKI e la Commissione autenticano le chiavi iniziali. Rendere identificabili queste dipendenze soddisfa l'esigenza di trasparenza del modello, ma non riduce automaticamente i poteri dei soggetti da cui dipendono.

3.3 - Analisi rispetto al threat model

3.3.1 - Attacchi di rete e correlazione dei dati

Intercettazione e modifica (1.5.1). I canali sicuri assunti nel modello proteggono il contenuto delle comunicazioni, comprese le credenziali istituzionali. L'attaccante di rete può comunque osservare i metadati e ostacolare la consegna. Le modifiche a richieste già firmate e l'iniezione di richieste prive delle firme necessarie vengono respinte; i replay restano soggetti allo stato del seriale, come discusso in 3.2. La soppressione può impedire il voto o il recupero della ricevuta: la mancata conferma non dimostra da sola chi abbia bloccato il traffico e un nuovo invio non assicura la consegna. La disponibilità contro DoS rimane esclusa dal WP1.

Avversario de-anonimizzatore (1.5.2). Un gestore che osserva i dati della propria fase incontra i limiti di ricostruzione passiva descritti in 3.2. Il Collector non è un passaggio riservato indispensabile perché pubblica seriale e richiesta. L'IdP è invece la fonte dell'associazione amministrativa identità-seriale. Questo non esclude che orari, IP o conoscenze esterne suggeriscano un'identità senza consultarlo: il protocollo non offre anonimato di rete. La ricostruzione diretta mediante dati dei ruoli va distinta sia da queste correlazioni sia dall'inferenza attiva del Mix analizzata sotto.

3.3.2 - Elettore, coercizione e dispositivo

Elettore disonesto (1.5.3). Con una sola credenziale, reinvii o nuove richieste firmate con lo stesso seriale non aumentano i voti registrati da un Collector onesto. Una busta malformata può consumare il seriale senza produrre un voto valido, ma non consente di sostituire la richiesta già accettata. Quanto al ripudio, la firma documenta l'uso della chiave pseudonima; non rende pubblicamente dimostrabile, da sola, chi sia lo studente. L'associazione conservata dall'IdP e la custodia della chiave restano necessarie per discutere un'attribuzione personale.

Coercizione e compravendita (1.5.4). Il terzo che ottiene la collaborazione dell'elettore può verificare la scheda preparata con le informazioni descritte in 3.2. Se controlla il dispositivo o osserva il voto, le firme possono attestare proprio la richiesta imposta. La ricevuta priva di preferenza non neutralizza questo scenario; l'assenza di rivoto impedisce inoltre di annullarne gli effetti dopo la prima accettazione. Restano i limiti dichiarati dal WP1, senza attribuire al sistema una resistenza completa alla coercizione.

La compromissione del dispositivo nel momento della scelta viola l'ipotesi esplicita di WP1 §1.9. Se avviene durante la successiva preparazione o prima dell'invio, l'ipotesi limitata al momento della scelta non la esclude. In entrambi i casi vale il limite sul cast-as-intended discusso in 3.2: i controlli possono confermare fedelmente una richiesta manipolata prima della protezione crittografica.

3.3.3 Collector, Mix e Commissione

Compromissione parziale del server (1.5.5). Chi altera il registro senza le chiavi pertinenti non può aggiornare validamente le firme. Un Collector disonesto dispone invece della propria chiave: può rigenerare voci e radici, ma non le firme degli elettori. Le ricevute conservate e il confronto delle viste permettono di rilevare omissioni successive all'accettazione o attestazioni incompatibili. Una vista isolata può tuttavia risultare internamente coerente, mentre la censura prima della Receipt resta possibile. Anche *accepted_at* è una dichiarazione del Collector: l'audit ne controlla l'appartenenza alla finestra, senza provare indipendentemente che non sia stato retrodatato.

Autorità finali disoneste (1.5.6). Un Mix può attribuire un ingresso valido agli errori esterni, oppure sostituire le uscite mantenendone il numero. Può produrre nuove buste interne valide usando la chiave pubblica di scrutinio: AES-GCM non autentica qui l'elettore originario. Il registro resta firmato e inalterato, mentre il Mix firma una lista diversa; l'audit non prova la corrispondenza uno-a-uno tra le due fasi.

La sostituzione può colpire anche la segretezza. Il Mix conserva una busta interna bersaglio valida e sostituisce le altre con cifrature fresche di No, pubblicando n uscite e zero errori esterni. Una Commissione onesta decifra questa lista e il totale dei Sì, pari a 0 oppure 1, rivela la preferenza del bersaglio. Hash, firme e quadrature possono risultare tutti coerenti; le cifrature fresche superano anche il controllo diagnostico delle buste duplicate. Il Mix ottiene così seriale-voto per il bersaglio senza la collusione della Commissione; aggiungendo l'IdP ottiene identità-voto. Questo attacco attivo supera la semplice unione dei dati e mostra che il Mix disonesto può apprendere una preferenza pur senza decifrare direttamente la busta interna.

Per la Commissione, una sola quota Shamir non ricostruisce K_T ; due quote sbloccano la chiave RSA completa. Il beneficio riguarda la custodia distribuita, senza garantire l'eliminazione di copie create durante il setup o la protezione della chiave dopo lo sblocco. Due trustee collusi possono anche approvare un conteggio falso, distribuendo diversamente No, Sì ed errori interni senza violare le quadrature. Le loro firme rendono attribuibile l'approvazione, ma non provano né le decifature né l'effettivo impiego di due quote. Un risultato con la firma di un solo trustee, invece, non supera il controllo del quorum.

Se al più un trustee è disonesto, due firme sullo stesso risultato includono quella di almeno un trustee onesto. I controlli previsti dal WP2 vincolano allora i conteggi alla lista L effettivamente elaborata. Questa garanzia dipende dalla verifica dei trustee e non dimostra che il Mix abbia preservato le schede della bacheca.

3.4 - Valutazione delle alternative e dei compromessi

Shuffle verificabile. Uno shuffle verificabile ridurrebbe la fiducia nel Mix, permettendo di controllare la corrispondenza fra ingressi e uscite senza pubblicare la permutazione. Nel protocollo a doppia busta occorrerebbe dimostrare anche la corretta apertura esterna e gestire gli errori: non basterebbe aggiungere una firma. Servirebbero strumenti crittografici ulteriori, dati di prova e lavoro di verifica.

Prove pubbliche di decifratura. Le prove pubbliche delle decifature ridurrebbero invece la fiducia nella Commissione, ma richiederebbero un meccanismo compatibile con lo scrutinio ibrido, compresa la validità delle schede. Sono interventi su due lacune differenti, con costi progettuali e computazionali.

Decifratura a soglia. Una vera threshold decryption eviterebbe la ricostruzione della chiave privata completa durante lo scrutinio. Rispetto alla condivisione di K_T richiederebbe generazione o distribuzione delle chiavi e operazioni di decifratura cooperative più articolate. Da sola non renderebbe verificabile lo shuffle e non proverebbe automaticamente il risultato pubblico. La custodia Shamir attuale mantiene invece semplice lo sblocco e tollera l'indisponibilità di un membro, a condizione che gli altri due dispongano di quote corrette; resta esposta alla loro collusione.

Rivoto. Il rivoto potrebbe sovrascrivere una scelta estorta se l'elettore ottenesse un momento libero prima della chiusura. Introdurrebbe però una regola di selezione fra più richieste, una storia delle sostituzioni e verifiche che non ne rivelino i collegamenti. *FIRST ACCEPTED* evita questa gestione, al prezzo della perdita di tale mitigazione e dell'impossibilità di correggere un invio già accettato.

Firma cieca. Il rilascio mediante firma cieca potrebbe invece ridurre il legame identità-seriale noto all'IdP, ma cambierebbe il rilascio della credenziale, il suo legame con la chiave temporanea e le dipendenze di fiducia. Non eliminerebbe i metadati né le manipolazioni del Mix.

Checkpoint firmati. Checkpoint firmati anticiperebbero i riferimenti con cui confrontare il registro, aumentando stato pubblicato e controlli durante la raccolta. Da soli non escluderebbero viste differenti; per confrontarle sistematicamente servirebbero testimoni o ulteriori comunicazioni, con nuove dipendenze e costi. L'unica radice finale mantiene invece la verifica di inclusione $O(\log n)$ per $n \geq 2$, rinviandola alla chiusura; i casi zero e uno sono definiti nel WP2. Il vincolo di efficienza del WP1 motiva il confronto fra questi costi, ma non costituisce una dimostrazione di usabilità: il WP2 stima $O(n)$ per le fasi complessive e non ne misura le latenze.

Diagnostica delle buste duplicate. Un affinamento più contenuto è la segnalazione di buste interne identiche nella lista pubblicata, prevista come controllo facoltativo nel WP2. Richiede soltanto confronti o hash dei dati già disponibili, senza nuove primitive, con lavoro lineare atteso e memoria aggiuntiva per i confronti. Può evidenziare sostituzioni per copia, ma non attribuirle con certezza al Mix, perché anche elettori collusi potrebbero aver preparato buste identiche; non rileva inoltre le sostituzioni con cifrature fresche. Il suo costo contenuto migliora la diagnosi delle anomalie, senza introdurre scarti automatici né fornire una prova della correttezza del mescolamento.

WP 4: Implementazione e prestazioni

Il referendum binario del WP2 è realizzato in un prototipo che permette di seguire la consultazione dalla votazione alle verifiche finali. Per descriverne il funzionamento, il capitolo affianca alle scelte implementative le prove eseguite e le misure raccolte sulle diverse fasi.

4.1 - Scelte implementative e architettura

La simulazione viene eseguita localmente in Python, dove IdP, Collector, Mix, Commissione e trustee operano come oggetti distinti con le rispettive chiavi e uno stato mantenuto in memoria. Il loro scambio di messaggi avviene tramite chiamate fra oggetti, che rappresentano i canali sicuri assunti dal protocollo.

L'accesso degli elettori passa attraverso account locali con password di prova, che simulano l'autenticazione istituzionale prevista nel modello. Per verificare le chiavi degli altri ruoli, i partecipanti partono dalla chiave pubblica di firma del manifesto, fornita come riferimento fidato, e controllano il manifesto che le associa ai rispettivi titolari.

L'apertura e la chiusura della raccolta seguono un orologio simulato comune a tutti i ruoli, che il comando di chiusura fa avanzare fino all'istante previsto dal manifesto. Le prestazioni vengono misurate con un contatore separato, così da rilevare la durata effettiva delle operazioni senza dipendere dall'avanzamento del tempo simulato.

4.2 - Struttura dei moduli e del codice

Il codice è suddiviso secondo le responsabilità del protocollo, lasciando ai moduli degli attori la gestione dei singoli passaggi e raccogliendo in funzioni condivise le operazioni che ricorrono in più fasi.

4.2.1 - Primitive e messaggi

Per applicare in modo uniforme i parametri del WP2, le operazioni di firma, hash e cifratura ibrida sono raccolte in *crypto_utils.py*, che usa *cryptography* per RSA-PSS, RSA-OAEP e AES-GCM e *hashlib* per SHA-256. Qui si trovano anche la codifica e le funzioni Shamir impiegate per suddividere e ricostruire la chiave che protegge la chiave privata di scrutinio. Le operazioni che richiedono casualità, compresa la permutazione del Mix, ricorrono alle sorgenti crittografiche del sistema.

In *models.py*, le *dataclass* descrivono i campi dei messaggi e includono i controlli sulla struttura dei dati, eseguiti prima dell'uso da parte degli attori. Per gli oggetti firmati, *signing_data()* individua i dati su cui calcolare la firma, mentre l'immutabilità degli oggetti e l'uso di tuple per le sequenze impediscono di modificare accidentalmente quanto già registrato.

Per ricostruire gli stessi byte durante le verifiche, messaggi, AAD e contesti di firma e hash vengono serializzati dalla funzione comune *enc*. I messaggi diventano oggetti JSON con campi nominati, codificati in UTF-8 con chiavi ordinate e separatori senza spazi, mentre le sequenze mantengono il proprio ordine. Firme, hash e altri dati binari usano Base64URL senza padding, applicata anche alle chiavi pubbliche.

4.2.2 - Attori e consultazione

Il modulo *actors.py* raccoglie le operazioni di IdP, elettore, Mix, Commissione e trustee. Durante l'accesso, l'IdP confronta i valori derivati dalle password con *scrypt* e un salt casuale per account; ottenuta la credenziale, l'elettore la conserva insieme alla chiave temporanea per preparare la richiesta

di voto. Dopo la raccolta interviene il Mix, che apre le buste esterne, conteggia gli errori di apertura e prepara la lista permutata delle buste interne.

Dopo aver controllato i dati pubblicati, la Commissione recupera la chiave di scrutinio dalle quote dei trustee partecipanti e procede all'apertura e al conteggio delle schede. Ciascun trustee ricalcola il risultato attraverso una routine condivisa con la Commissione e lo confronta con quello proposto, apponendo la firma solo se i due coincidono.

Il passaggio fra raccolta, chiusura, Mix e scrutinio è coordinato da *ElectionEnvironment*, in *election.py*, che prepara anche le chiavi e il manifesto della consultazione. Per ogni fase, il risultato viene conservato solo quando l'operazione è terminata, mentre le richieste di ripetere Mix o scrutinio vengono respinte se la fase è già stata completata.

4.2.3 - Raccolta e verifiche

Il Collector di *board.py* conserva le registrazioni e le ricevute originali, in modo da restituirle quando riceve un reinvio identico, anche dopo la chiusura. Durante l'accettazione, il controllo del seriale e l'aggiornamento dello stato avvengono sotto lo stesso lock, impedendo che due richieste concorrenti vengano entrambe accettate con la stessa credenziale. Le firme della voce e della ricevuta vengono preparate prima di modificare la raccolta, così che un errore in questo passaggio non lasci una registrazione incompleta.

A partire dalle registrazioni, *board.py* costruisce l'albero Merkle e le prove usate nelle verifiche di inclusione. Queste verifiche, insieme a quelle sulle ricevute e sui dati pubblicati, sono raccolte nella classe *UniversalVerifier* di *audit.py*, a cui si appoggiano anche Mix, Commissione e trustee per applicare le stesse regole sui riferimenti e sui conteggi.

4.2.4 - Interfaccia e sperimentazione

Attraverso il menu di *main.py* si può seguire l'intera consultazione, dall'inizializzazione e dall'invio dei voti fino al recupero delle registrazioni e alle verifiche, scegliendo separatamente quando eseguire Mix e scrutinio. Il sottomenu degli scenari permette di provare i controlli sui dati della consultazione corrente: segnala i prerequisiti mancanti e applica le alterazioni a copie dei messaggi, preservando gli originali. Per le misurazioni, *benchmark.py* richiama le stesse classi senza passare dall'interazione con il menu.

4.3 - Sperimentazione e verifica del funzionamento

4.3.1 - Esecuzione del flusso completo

Per provare il flusso completo sono stati usati i tre account del menu, da *studente-001* a *studente-003*, con la password *aps-demo*, esprimendo nell'ordine No, Sì e No. A ciascun invio il Collector ha associato una registrazione, agli indici 0, 1 e 2, e una ricevuta che l'elettore ha poi verificato.

Il recupero della prima richiesta ha permesso di ritrovare la registrazione e la ricevuta originali, mentre dopo la chiusura le tre prove di inclusione sono state verificate rispetto alla radice finale. Elaborando la bacheca, il Mix ha prodotto tre buste interne dalle quali lo scrutinio ha ricavato un Sì e due No, senza errori esterni o interni, come riportato nell'estratto seguente.

Estratto 4.1. Risultato dell'esecuzione con tre elettori

```
Mix: 3 buste interne pubblicate, 0 errori esterni.  
Commissione e trustee: Sì 1, No 2, non valide 0.  
Verificatore: controlli pubblici completati.  
manifest: valid  
board_and_merkle_root: valid  
accepted_ballots: 3  
mix: internally-consistent  
tally: internally-consistent  
duplicate_inner_envelopes: 0
```

Nell'esecuzione, la verifica individuale ha confermato l'inclusione delle registrazioni nella bacheca finale. Gli esiti *internally-consistent* riguardano invece l'audit pubblico del Mix e dello scrutinio, che

ha controllato le firme e la coerenza dei riferimenti e dei conteggi, con le garanzie e le assunzioni discusse nel WP3.

4.3.2 - Scenari di rifiuto

Sulla stessa consultazione sono stati poi eseguiti gli otto scenari del menu, richiamando per ciascun tentativo il controllo effettivo del ruolo coinvolto. In tutti i casi è stato ottenuto il rifiuto atteso, mentre il confronto dei dati pubblicati prima e dopo le prove ha confermato che lo stato valido della consultazione era rimasto invariato.

Tabella 4.1. Scenari eseguiti sulla consultazione corrente

Operazione	Controllo richiamato	Esito osservato
Password errata	Verifica delle credenziali presso l'IdP	Autenticazione rifiutata
Seconda richiesta con lo stesso seriale	FIRST_ACCEPTED e confronto con l'originale	Nuova richiesta rifiutata
Richiesta alterata	Firma della CastRequest	Firma non valida
Ricevuta alterata	Firma della Receipt	Firma non valida
Prova Merkle alterata	Numerosità rispetto alla radice firmata	Prova rifiutata
Record del Mix alterato	Hash della lista pubblicata	Record incoerente
Risultato alterato	Riferimento al MixRecord	Riferimento non valido
Scrutinio con un solo trustee	Numero di trustee partecipanti	Quorum insufficiente

Nello scenario della ricevuta è stato modificato l'hash della voce nel corpo del messaggio, lasciando la firma originale: la verifica ha così rilevato l'alterazione e prodotto il rifiuto riportato nell'estratto.

Estratto 4.2. Verifica di una ricevuta alterata

Elettore: verifica la firma di una ricevuta alterata.
Rifiuto atteso: Firma non valida: Receipt

4.3.3 - Test automatici

La verifica del funzionamento comprende anche i 18 test automatici basati su *unittest*, tutti superati, che esercitano codifica e primitive insieme al flusso completo e alla gestione dello stato. Le prove considerano alterazioni dei dati, consultazioni vuote, errori nelle buste esterne e interne, reinvii, richieste concorrenti e limiti temporali della raccolta, includendo il caso in cui un trustee deve rifiutare un risultato diverso dal proprio conteggio anche se le somme complessive sono coerenti. Gli esiti sono riportati in *results/test_results.txt*.

4.4 - Valutazione delle prestazioni

4.4.1 - Metodo di misurazione

Le prestazioni sono state misurate su cinque consultazioni indipendenti per ciascun carico di 10, 50 e 100 elettori, generando ogni volta nuove chiavi e nuovi account. In ciascuna esecuzione sono stati alternati No e Sì per confrontare lo scrutinio con un conteggio noto, ottenendo in tutte le 15 prove il risultato atteso con tutti i voti validi.

Per misurare la durata delle operazioni si usa *perf_counter_ns*, convertendo il tempo trascorso in millisecondi. Le attività individuali vengono prima riassunte con una media per elettore all'interno di ogni consultazione, in modo da ricavare poi media e deviazione standard campionaria dai cinque valori ottenuti. Per le fasi collettive, lo stesso riepilogo si applica ai cinque tempi complessivi. I timer comprendono l'esecuzione delle operazioni, lasciando fuori interazione con il menu, stampa, controllo del conteggio atteso, calcolo delle dimensioni e scrittura dei risultati.

Nella configurazione rientrano la generazione delle chiavi delle autorità, il manifesto, le quote e la preparazione degli account con gli hash delle password. Quando viene creato un elettore, il timer comprende la verifica del manifesto e la generazione della sua chiave temporanea; nel tempo di autenticazione rientrano poi il controllo della password, l'emissione e la verifica della credenziale. Il tempo di accettazione arriva fino alla produzione delle firme della voce di bacheca e della ricevuta.

Alla chiusura, la misura comprende l'avanzamento dell'orologio e la costruzione dell'albero con la firma della radice. Da quel momento si misurano separatamente la generazione delle prove, che ricostruisce l'albero a ogni richiesta, e la verifica individuale, eseguita su una prova già disponibile. Il tempo del Mix copre i controlli, le aperture esterne, la permutazione e la firma del record, mentre quello dello scrutinio comprende la ricostruzione della chiave, il conteggio della Commissione e il ricalcolo dei due trustee firmatari. I controlli ripetuti durante queste fasi restano inclusi nei rispettivi tempi.

4.4.2 - Risultati del benchmark

Nelle Tabelle 4.2 e 4.3, i tempi delle cinque consultazioni di ogni carico sono riassunti come media $\pm$ deviazione standard campionaria, distinguendo le operazioni riferite a un singolo elettore dalle fasi misurate sull'intera consultazione. Poiché i tempi dipendono dalla macchina e dalle versioni software utilizzate, i risultati si riferiscono alle condizioni in cui sono state svolte le prove.

Tabella 4.2. Tempi medi per elettore in millisecondi

Operazione	10 elettori	50 elettori	100 elettori
Creazione dell'elettore e chiave temporanea	57,000 $\pm$ 11,561	51,775 $\pm$ 6,293	52,173 $\pm$ 1,708
Autenticazione e credenziale	39,892 $\pm$ 0,550	39,863 $\pm$ 0,690	40,370 $\pm$ 0,635
Preparazione della richiesta	1,647 $\pm$ 0,128	1,612 $\pm$ 0,050	1,631 $\pm$ 0,024
Accettazione e ricevuta	1,920 $\pm$ 0,087	1,881 $\pm$ 0,077	1,891 $\pm$ 0,058
Verifica immediata della ricevuta	0,449 $\pm$ 0,023	0,431 $\pm$ 0,029	0,435 $\pm$ 0,020
Generazione della prova Merkle	0,254 $\pm$ 0,014	1,093 $\pm$ 0,084	2,193 $\pm$ 0,079
Verifica individuale	0,755 $\pm$ 0,069	0,767 $\pm$ 0,060	0,847 $\pm$ 0,038

Tabella 4.3. Tempi delle fasi complessive in millisecondi

Operazione	10 elettori	50 elettori	100 elettori
Configurazione	850,806 $\pm$ 76,159	2.448,992 $\pm$ 65,744	4.428,628 $\pm$ 335,070
Chiusura e radice finale	0,998 $\pm$ 0,281	1,626 $\pm$ 0,043	2,758 $\pm$ 0,182
Mix	16,424 $\pm$ 0,864	68,269 $\pm$ 7,422	138,324 $\pm$ 7,801
Scrutinio e controlli dei trustee	83,021 $\pm$ 5,176	234,711 $\pm$ 14,608	450,865 $\pm$ 48,362
Audit pubblico	6,731 $\pm$ 0,639	28,930 $\pm$ 1,686	61,610 $\pm$ 5,723

Per confrontare le dimensioni degli oggetti, si contano i byte del JSON effettivamente prodotto da *enc*, aggregando poi i valori sulle cinque consultazioni nella Tabella 4.4. Gli oggetti individuali sono rappresentati dalla media per elettore, mentre per gli altri si misura l'oggetto completo. La prova Merkle comprende indice, numerosità e percorso; per il Mix sono riportate sia la dimensione del record completo di lista sia quella della sola lista delle buste.

Tabella 4.4. Dimensioni degli oggetti in byte

Oggetto	10 elettori	50 elettori	100 elettori
Manifesto firmato	3.971,00	3.971,00	3.971,00
Credenziale, per elettore	864,00	864,00	864,00
Richiesta, per elettore	2.488,00	2.488,00	2.488,00
Voce di bacheca, per elettore	3.064,00	3.064,80	3.064,90
Ricevuta, per elettore	535,00	535,80	535,90
Radice finale firmata	504,00	504,00	505,00
Prova Merkle, per elettore	313,50	453,14	523,74
Record del Mix con lista	5.599,00	25.599,00	50.601,00
Lista delle buste interne	5.001,00	25.001,00	50.001,00
Risultato firmato	1.017,00	1.019,00	1.020,00
Bacheca completa	30.651,00	153.291,00	306.591,00

I valori delle tabelle sono ricavati dai campioni di *results/benchmark.csv* e dal riepilogo di *results/benchmark_summary.csv*, accompagnati dalle versioni software e dalle convenzioni di misura registrate in *results/benchmark_environment.json*.

4.4.3 - Discussione dei risultati

Il costo per elettore rimane vicino a 1,6 ms per preparare la richiesta e a 1,9 ms per accettarla, mentre la creazione dell'elettore e l'autenticazione richiedono tempi maggiori. Le loro medie, rispettivamente fra 52 e 57 ms e intorno a 40 ms, comprendono la generazione della chiave temporanea e le operazioni legate alla password e alla credenziale. La preparazione degli hash delle password incide anche sulla configurazione, che aumenta con il numero degli account.

Con l'aumentare delle schede da elaborare crescono i tempi complessivi di Mix, scrutinio e audit. Lo scrutinio arriva a circa 451 ms con 100 elettori, includendo anche il ricalcolo dei due trustee, mentre la verifica individuale varia più contenutamente, da circa 0,75 a 0,85 ms. La generazione delle prove cresce maggiormente perché, a ogni richiesta, ricostruisce l'intero albero Merkle.

L'aumento degli elettori si riflette soprattutto sulle dimensioni della bacheca e della lista del Mix, che raccolgono un numero maggiore di registrazioni e buste, mentre la singola richiesta resta di 2.488 byte. Anche la prova Merkle cresce, passando in media da 313,50 a 523,74 byte; negli altri oggetti, le piccole variazioni dipendono dalle cifre necessarie a rappresentare indici e conteggi. Il confronto riguarda i carichi della simulazione eseguiti, con le assunzioni e la portata delle verifiche già discusse nel WP3.